Expressions and Algorithms

Last modified: 2026-10-15 13:56:31 (PDT)

Remark

Remark. A function can be written down as an expression and computed by an algorithm. This page defines expressions, which write down the functions in the Morrison Lab's probability and statistics notes, their parse trees, and algorithms, which compute those functions.

Expressions

Definition 0.1 (Expression)**.** An **expression** is a string of symbols, built from constants, variables, operations and function names according to the rules of the notation. When its variables are read as inputs, an expression *denotes* a function: it fixes the output for each input.

Example 0.1 (Two expressions for one function)**.** The expressions $ax^2 + bx + c$ and $(ax + b)\,x + c$ denote the same function of x for any fixed a, b and c, because they give the same value at every x.
Precedence parses $ax^2 + bx + c$ as a sum of three terms: the power x^2 is applied first, then a multiplies x^2 and b multiplies x, and the three products are added. The parentheses in $(ax + b)\,x + c$ force a different tree: a multiplies x, b is added, the sum multiplies x, and c is added.

Definition 0.2 (Parse tree)**.** The **parse tree** of an expression is the tree that shows how its symbols group into operations. Its leaves are the constants and variables of the expression, and each internal node is an operation applied to the nodes below it.

Example 0.2 (The parse tree of a polynomial)**.** The parse tree of $(ax + b)\,x + c$, with each operation having two children, is:

- the root $+$, whose children are
  - the node $\times$, whose children are
    - the node $+$, whose children are
      - the node $\times$, whose children are the leaves a and x,
      - the leaf b;
    - the leaf x;
  - the leaf c.

Read from the leaves up, the tree says to multiply a by x, add b, multiply by x, and add c. The expression $ax^2 + bx + c$ has a different tree. Its root is the $+$ whose children are $ax^2 + bx$ and c, and the first of these is a $+$ whose children are ax^2 and bx.

Remark (Expressions, precedence and parse trees)

Remark 0.1 (Expressions, precedence and parse trees). An expression does not say how to compute the output it fixes, and many expressions denote the same function. A function is the input-output relationship itself; an expression is one way of writing it down.
The rules of operator precedence (PEMDAS: parentheses, exponents, multiplication and division, addition and subtraction) decide how the symbols group. They turn the string into its parse tree (Definition 0.2).

Algorithms

Definition 0.3 (Algorithm)**.** An **algorithm** is a finite list of unambiguous steps that, applied to any valid input, halts and returns an output. An algorithm *computes* a function: for each input it returns the function's value at that input.

Example 0.3 (Two algorithms for one function)**.** Take the function that Example 0.1 writes two ways. Evaluating the tree of $ax^2 + bx + c$, with x^2 computed as $x \cdot x$, takes three multiplications and two additions. Evaluating the tree of Horner's form $(ax + b)\,x + c$ takes two multiplications and two additions. At $a = 2$, $b = -3$, $c = 5$ and $x = 1.5$, the first algorithm computes $4.5 - 4.5 + 5 = 5$ and the second computes $(3 - 3) \cdot 1.5 + 5 = 5$, the same value by different steps.

Remark (How functions, expressions and algorithms relate)

Remark 0.2 (How functions, expressions and algorithms relate). The three ideas fit together this way:

- A function is the input-output relationship.
- An expression denotes a function, and many expressions denote the same one.
- An algorithm computes a function, and many algorithms compute the same one.

An expression is not an algorithm, because it does not say what to do, or in what order. Given values for its variables, evaluating its parse tree from the leaves up is one algorithm for the function it denotes, and a different expression for the same function gives a different tree and a different algorithm.

Remark (Values with no known algorithm)

Remark 0.3 (Values with no known algorithm). Some functions have no known finite algorithm for their exact values, and expressions that denote them, such as a limit, a supremum, an integral with no closed form, or an $\arg\min$ over a set with no formula for the minimizer, cannot be evaluated by following their tree. Such a value can still be well defined and approximated by an algorithm.

Example 0.4 (A minimizer with no known formula)**.** The minimizer $\arg\min_{x\in\mathbb{R}}(x-\cos x)^2$ is well defined, because $x-\cos x$ is strictly increasing and equals zero at one point, yet that point has no known closed-form expression, so no known finite list of arithmetic steps returns it exactly. An algorithm such as bisection approximates it as closely as we like.